Implementing a Trusted Environment for the use of Untrusted Digital IPs

Kristian Nikolov (kdn2117), Jayden Lee-Sin (jcl2257)
github.com/mthrndr/COMS-6424-Final/

April 2026

1 Introduction

In the production of secure devices, such as satellites or secure communication devices, third-party digital intellectual property (IP) is often used to speed up production. By reducing the number of custom hardware components that are integrated into the device, the development time is significantly reduced. As a result, producers of these devices often integrate common components, such as a CPU core, by purchasing the associated HDL code and placing it within a larger ASIC or within the fabric of an FPGA. However, the purchase of these IPs in these environments is a massive security risk, as it is costly and time consuming to validate an IP and ensure there is no potentially malicious behavior. To address this, we have developed a system that integrates two untrusted IPs using dual modular redundancy (DMR) to create a trusted environment, mitigating the security risk posed by the integration of these untrusted IPs. To utilize our solution, a security minded group must source two third-party IPs which are architecturally identical, but may differ micro-architecturally, or implement different feature sets as long as only the shared feature set is used for the secure application.

Although the general methodology of this system can be broadly applied to untrusted third-party IPs, we have implemented it specifically to create a trusted environment surrounding two RISC V 32-bit CPU cores, both produced by the OpenHW Foundation [1], a not-for-profit global organization that develops open source hardware. The two selected cores, the CV32E40X [2] and the CV32E40X [3], are sister cores, belonging to the Core-V family, which is a project centered around the creation of open source RISC V cores. They both feature four stage in order pipelines, use the Open Bus Interface (OBI), and are relatively architecturally similar, which is crucial to our solution. Additionally, the use of the same bus protocol makes the development of our system much easier, as interlinks between the cores and a given component are identical, and data can be expected in a common format. Despite this relation, the two cores were developed separately, being managed by the same members of the OpenHW Foundation, but contributed to and built by different authors, and implementing different feature sets, thus providing a valid point of comparison.

The CV32E40X (X-core) implements the extension interface (CV-X-IF), which allows the processor to connect custom co-processors to expand the RISC V ISA. It claims to feature basic security protocols, such as physical memory attributes (PMA), but focuses on the CV-X-IF, which it claims to allow connected co-processors to read and write to the core's register file at a low latency. It is considered depreciated or inactive by the OpenHW Foundation.

Meanwhile, the CV32E40S (S-core) implements a security focus. Supporting physical memory protections (PMP) as well as a PMA, on top of utilizing common security practices such as dummy instructions, parity signals, and various checksums. The core additionally boasts security alerts, data independent timing, register file error correction, a hardened program counter, bus protocol hardening, a hardened control/status register (CSR), and more.

These feature sets create timing differences, and massive disparities between

the RTL implementations of the devices, making them architecturally identically, but unique from a micro-architectural perspective and maintained independently by different contributors, further separating the two cores as even previously identical components have been adjusted over the cores' nine year development cycles. To prove this point, the following data was gathered with the use of an LLM tool [4]:

Filename	Difference
cs_registers.sv	1,675 lines
include/pkg.sv	679 lines
load_store_unit.sv	484 lines
core.sv	392 lines
controller_fsm.sv	376 lines
id_stage.sv	373 lines
if_stage.sv	350 lines
alert.sv	Unique to S-core
dummy_instr.sv	Unique to S-core
lfsr.sv	Unique to S-core
obi_integrity_fifo.sv	Unique to S-core
pc_check.sv	Unique to S-core
pmp.sv	Unique to S-core
rchk_check.sv	Unique to S-core
register_file_ecc.sv	Unique to S-core
a_decoder.sv	Unique to X-core
align_check.sv	Unique to X-core
if_xif.sv	Unique to X-core

Table 1: Differences in RTL code between X-core and S-core

Meanwhile, only alu.sv, div.sv, mult.sv, popcnt.sv, ff_one.sv, prefetcher.sv, sleep_unit.sv, int_controller.sv, and if_c_obi.sv are identical between the two cores. These files are largely based on standard implementations, and have been edited hundreds of times before being pulled into the X-core and S-core projects at their inception. This large difference in design confirms that if a vulnerability or malicious behavior is inserted in either of the cores, it is unlikely to be shared. Due to the nature of our system, shared vulnerabilities cannot be detected, making it imperative to select architecturally identical but different IPs, ideally from different vendors and different contributors.

Additionally, we have elected to utilize the OpenHW Foundation's Core-V Verification Strategy [5]. This project provides verification tools allowing for easy comparison of both the S-core and X-core against known RISC-V architectural standards, including the ability to run various C programs. This will be used to verify the functionality of the cores, as well as a source of inputs to test the system once implemented.

2 Threat Model

Before explaining the system we aim to implement, we propose the following concrete threat model describing the attacks our system aims to mitigate. Our system works under the following assumptions:

- Assumption #1: The hardware is sufficiently isolated. This implies attackers lack direct access to the hardware physically, are incapable of building a sufficient side-channel, and lack access to anything connected to the hardware including where the hardware derives its power from and the hardware's connected peripherals.
- Assumption #2: Any external communication done by the hardware is secured, meaning that attackers have no ways to read data transmitted from the hardware to intended listeners, and no way to listen to communications intended for the hardware itself.

This limits the scope of our system to hardware backdoors. Attacks which are inserted at RTL development time and lay dormant in the system until a specific trigger occurs. This is a valid concern given the nature of the devices our system aims to protect, that being secured devices relied upon for communications, on edge processing, or other high priority applications. As a result of the function of these devices, nation states and state actors with the ability to insert hardware backdoors are significant threats. Additionally, these devices have extremely high budgets, specifically for security, given the nature of their operations, making our system a valid investment.

To mitigate these backdoors, our system aims to deny the common triggers used to initiate malicious behaviors. These triggers can be divided into two groups, time based triggers, and data based triggers [6].

Time based triggers use the runtime of the hardware to activate backdoors. Attackers build simple circuits, covertly inserted within the RTL of the hardware, that check the number of cycles a piece of hardware has been running for. After a set number of cycles, the backdoor activates, often placing the hardware in an illegal state, leaking data, or otherwise thwarting security measures. These backdoors are generally difficult to detect as they only activate after a large number of cycles, escaping common verification and validation flows which don't run long enough for the backdoor to trigger.

Data based triggers use sequences of uncommon bits to unlock backdoors. Attackers select a key, which when observed by a covertly placed circuit, either change the state of the hardware, leak memory, or force the execution of a specific program. This allows the backdoor a high level of control over the hardware, and is hard to detect as the search space for this key is extremely large. It is also hard to avoid, as the key is unknown and is often an uncommon instruction, so when the hardware attempts to run any program that contains this key, the backdoor is silently triggered.

3 Methodology & Setup

To defend against these triggers our system leverages DMR. Two architecturally identical cores are placed in parallel, receiving the same instructions and running the same program. As stated earlier it is critical that these cores are not identical, as shared data based backdoors cannot be detected by the system. These cores are then attached to the three IPs within the system. The reset control buffer, which thwarts time based attacks by resetting the cores every set amount of cycles, the external memory management unit (EMMU), which helps to prevent data based backdoors by comparing requests to memory by the cores, and the core monitor comparison unit, which compares the output of the cores to confirm that no illegal states or malicious behavior has begun. If a data based backdoor is detected, the EMMU blocks all memory access and the core monitor comparison unit raises a flag, indicating that the hardware can no longer be trusted. All of the RTL files can be found in the rtl/ directory, they have all also been synthesized in YoSys, scripts to repeat this synthesis can be found in the syn/ directory.

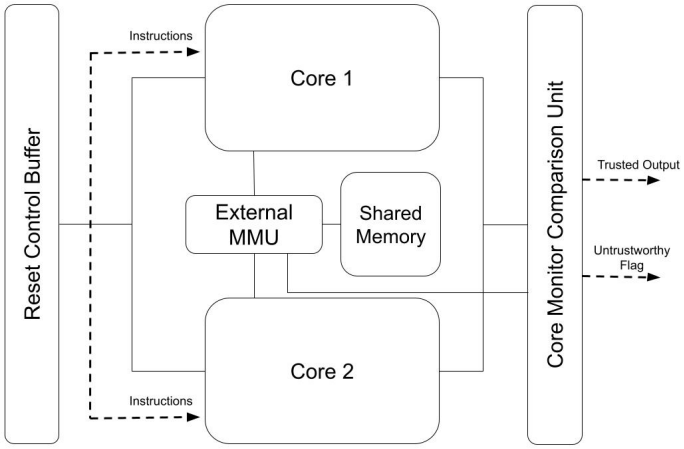

Figure 1: The system's architecture

3.1 Reset Control Buffer

The reset control buffer contains a parametrized counter (default set to 50,000) that triggers a reset of both cores. This thwarts time based attacks as a reset would also reset the number of observed cycles, effectively keeping the maximum number of cycles under 50,000, or another set value, which is too small to trigger most time based attacks. However, this would also prevents the core from running programs that would be longer than 50,000 cycles. So, in order

to maintain full functionality, the reset control buffer saves the current program counter of each core and resets the core with this location as the boot address, allowing a program to continue but still preventing time based attacks. At the RTL level, this IP is a simple FSM as illustrated below.

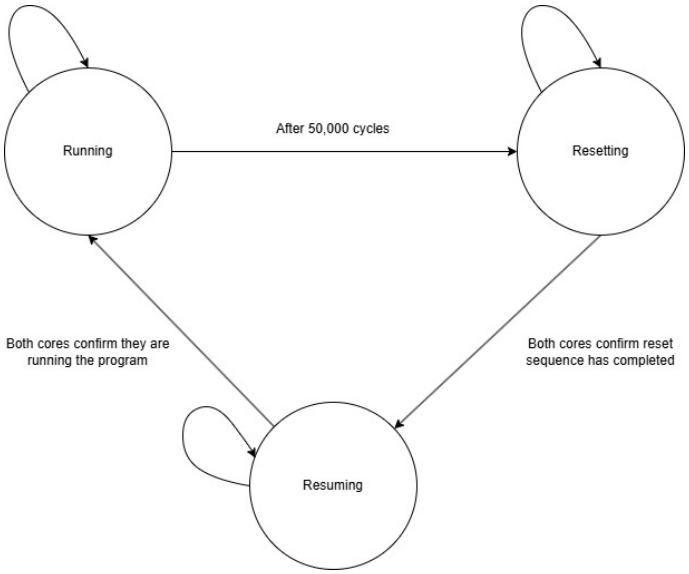

Figure 2: The Reset Control Buffer Modeled as an FSM

During the running phase, the IP checks if the counter is zero. If it is, the IP transitions to the resetting phase, otherwise the counter is decremented and the core continues normal functionality. In the resetting phase, the current program counter is saved and both cores are reset, then once the cores are finished resetting, the machine moves to the resuming phase. Here, the machine confirms that the cores have finished the boot sequence, afterwards, the counter is reset to it's assigned value, and the machine moves back to the running phase.

3.2 External Memory Management Unit

The EMMU compares memory requests from both cores and sits between the cores and their shared memory. It accepts one request from each of the cores and ensures that both requests match exactly, while accounting for the S-core's security features. Due to the micro-architectural differences in the cores, the EMMU uses two three deep FIFOs to create skew tolerance, stalling the requests of one core until the other core produces a request for comparison, and then compares the heads of the FIFOs. If the EMMU receives the flag from the core monitor comparison unit that either of the cores is untrustworthy, the IP raises its own untrustworthy flag and blocks all memory requests. This flag is sticky and will block memory requests until the entire system is reset. Similarly, if

two requests ever diverge, the IP raises its untrustworthy flag, alerting the core monitor comparison unit and blocking all memory requests. Additionally, as soon as the untrustworthy flag is raised, both FIFOs are flushed, so the EMMU refuses to observe any more requests. This combats data based backdoors which attempt to maliciously read or write to memory as the cores must make identical requests to be trusted. The EMMU is parametrized to support the data bus and the instruction bus, can tolerate up to two outstanding approved requests at once, and is a mix of sequential and combinational logic as illustrated below.

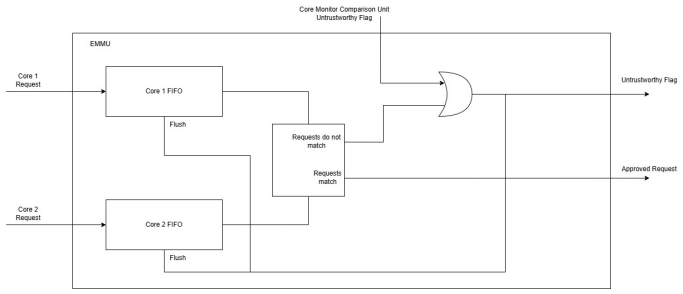

Figure 3: The EMMU modeled as a block diagram

3.3 Core Monitor Comparison Unit

The core monitor comparison unit uses the program counter and data writes to monitor the behavior of each core, ensuring they align. This looks for any divergence in the way the cores handle a program and raises an untrustworthy flag if a difference is detected. For the cores to remain trusted both the program counter and all of the components of data writes must be identical. This includes address, data to be written, and write enable signals. Similarly to the EMMU, the mismatch flag is sticky and infinitely stalls the cores until reset once raised. It is also skew tolerant similar to the EMMU and uses 16 deep FIFOs for both the program counter and data writes.

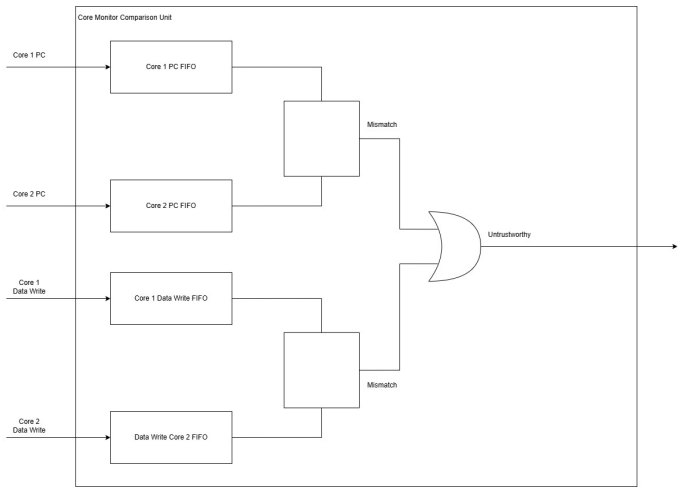

Figure 4: The Core Monitor Comparison Unit modeled as a block diagram

4 Verification & Validation

The described system relies on both the cores functioning properly, as well as the functionality and security of the system's own IPs. To verify this we implemented a rigorous verification pipeline that tested each core individually, then each IP individually for functionality and security. This ensures that the system functions correctly, and that the custom hardware it introduces does not include backdoors. On top of the IPs being verified, the FIFO submodule instantiated by the core monitor comparison unit and the EMMU was also verified individually.

4.1 Architectural

As previously stated, to confirm the architectural validity of both cores we leveraged the OpenHW Foundation's Core-V Verification Strategy. This project provides a framework for running not only the RISC-V architectural tests, but also various C programs, providing both functional and code coverage. This allowed us to confirm that both cores are fully functional and meet the RISC-V standards, as the verification environment has built in tools that compare each core's output to an architectural golden model for known test vectors. Each core was verified individually before the construction of our custom IPs. Architectural verification can be ran from the main directory using the commands 'make test-x', 'make test-s' and 'make test-dual'.

4.2 Functional

After the development of each custom IP, they were individually verified for functionality. This process involved building System Verilog test benches for each IP. These test benches run through each of the cases considered during development using direct testing. This includes both matches and intentional mismatches to verify that the untrustworthy flags for both the EMMU and the core monitor comparison unit are raised consistently at the correct time. These test benches can be found in the sim/functional/ directory. Functional verification can be run individually using the makefile within the sim/functional/ directory, or all of the IPs can be verified from the main directory using 'make test-ip-functional'.

4.3 Security

After functional verification, each IP was verified for security, aiming to eliminate any backdoors or bugs mistakenly inserted in the IPs during development that could not be detected by functional verification. This was done initially by a code review of each IP. Then, security test benches were developed. These test benches use System Verilog Assertions (SVA) along with 10,000 constrained random (CRV) tests per IP. Each of these test benches was run a minimum of

10 times. The large amount of random tests also uses cover properties to ensure that all corner cases are hit during simulation. The SVA, CRV, and cover properties allow us to achieve 100% line coverage, branch coverage, and FSM coverage (where applicable), confirming that no backdoors exist within the custom IPs surrounding the cores.

5 Conclusion

Following development and verification, the system and cores were combined into a top verilog module. The complete system was then run using the same architectural tests mentioned before. When running with 'make test-dual TEST=hello-world' we observed the cores running in tandem for 9 cycles with the same output to memory, before in the 10th cycle the s core raised an integrity fault "mcause=00000019". Then in the 11th cycle, this causes an actual difference in the output of the cores, as seen in the addresses sent to the instruction memory. Compared to the standard hello-world seen in Figure 5 test for the cores, they seem to be running fine up until this point. Note the address sequence of 88, 8c, then 90 in the original. In Figure 6 we see that both cores follow this sequence, with a pause for timing, but the x core finishes with 90 whereas the s core finished with 00. Unfortunately we were not able to resolve this issue; however, this proves that the overall setup is nearly functional, and we can accurately catch discrepancies between the cores.

```
[tb_top_verilator] finished dumping memory
[0] cyc=0 pc=00000004 instr_req=0  instr_gnt=1 data_req=0 we=0 daddr=00000000 slp=0 mcause=00000000 mepc=00000000
[50] cyc=1 pc=00000004 instr_req=0  instr_gnt=1 data_req=0 we=0 daddr=00000000 slp=0 mcause=00000000 mepc=00000000
[60] cyc=2 pc=00000080 instr_req=1  instr_gnt=1 data_req=0 we=0 daddr=00000000 slp=0 mcause=00000000 mepc=00000000
[70] cyc=3 pc=00000084 instr_req=1  instr_gnt=1 data_req=0 we=0 daddr=00000000 slp=0 mcause=00000000 mepc=00000000
[80] cyc=4 pc=00000088 instr_req=1  instr_gnt=1 data_req=0 we=0 daddr=00000000 slp=0 mcause=00000000 mepc=00000000
[90] cyc=5 pc=0000008c instr_req=1  instr_gnt=1 data_req=0 we=0 daddr=00014080 slp=0 mcause=00000000 mepc=00000000
[100] cyc=6 pc=00000090 instr_req=1  instr_gnt=1 data_req=0 we=0 daddr=00014608 slp=0 mcause=00000000 mepc=00000000
[110] cyc=7 pc=00000094 instr_req=1  instr_gnt=1 data_req=0 we=0 daddr=00400088 slp=0 mcause=00000000 mepc=00000000
[120] cyc=8 pc=00000098 instr_req=1  instr_gnt=1 data_req=0 we=0 daddr=00400000 slp=0 mcause=00000000 mepc=00000000
```

Figure 5: Expected Instruction MMU Requests for hello-world

```
--- MMU Request Mismatch at 120 ---
Field    | X-Core (CV32E40X) | S-Core (CV32E40S)
---------|-------------------|-------------------
Addr:    | 00000088          | 00000088
WE:      | 0                 | 0
BE:      | 0                 | 0
WData:   | 00000000          | 00000000
Prot:    | 110               | 110
MemType: | 00                | 00
Dbg:     | 0                 | 0
-----------------------------------------------

[130] cyc=9 pc=0000008c instr_req=1 instr_req_s=0 instr_req_x=0 both_head_valid=1 heads_match=1 outstanding_room=1 mismatch_q=0 comp_untrust=0 x_empty=0 s_empty=0 x_pop=1 s_pop=1 flush_fifos=0 mismatch=0 mm_field_test=0 mm_integrity_set=0 s_req=0, reqp
ar_ok=1, achk_ok=1 instr_gnt=1 data_req=0 we=0 daddr=00000000 slp=0 mcause=00000000 mepc=00000000, fault_det_comp_o=0

--- MMU Request Mismatch at 130 ---
Field    | X-Core (CV32E40X) | S-Core (CV32E40S)
---------|-------------------|-------------------
Addr:    | 0000008c          | 0000008c
WE:      | 0                 | 0
BE:      | 0                 | 0
WData:   | 00000000          | 00000000
Prot:    | 110               | 110
MemType: | 00                | 00
Dbg:     | 0                 | 0
-----------------------------------------------

[140] cyc=10 pc=00000000 instr_req=0 instr_req_s=1 instr_req_x=1 both_head_valid=0 heads_match=1 outstanding_room=1 mismatch_q=0 comp_untrust=0 x_empty=1 s_empty=1 x_pop=0 s_pop=0 flush_fifos=0 mismatch=0 mm_field_test=0 mm_integrity_set=0 s_req=1, req
par_ok=1, achk_ok=1 instr_gnt=1 data_req=0 we=0 daddr=00000000 slp=0 mcause=00000019 mepc=00000080, fault_det_comp_o=0

--- MMU Request Mismatch at 140 ---
Field    | X-Core (CV32E40X) | S-Core (CV32E40S)
---------|-------------------|-------------------
Addr:    | 00000000          | 00000000
WE:      | 0                 | 0
BE:      | 0                 | 0
WData:   | 00000000          | 00000000
Prot:    | 000               | 000
MemType: | 00                | 00
Dbg:     | 0                 | 0
-----------------------------------------------

[150] cyc=11 pc=00000090 instr_req=0 instr_req_s=1 instr_req_x=1 both_head_valid=1 heads_match=0 outstanding_room=1 mismatch_q=0 comp_untrust=0 x_empty=0 s_empty=0 x_pop=0 s_pop=0 flush_fifos=1 mismatch=0 mm_field_test=1 mm_integrity_set=0 s_req=1, req
par_ok=1, achk_ok=1 instr_gnt=0 data_req=0 we=0 daddr=00000000 slp=0 mcause=00000019 mepc=00000080, fault_det_comp_o=0

--- MMU Request Mismatch at 150 ---
Field    | X-Core (CV32E40X) | S-Core (CV32E40S)
---------|-------------------|-------------------
Addr:    | 00000090          | 00000000
WE:      | 0                 | 0
BE:      | 0                 | 0
WData:   | 00000000          | 00000000
Prot:    | 110               | 110
MemType: | 00                | 00
Dbg:     | 0                 | 0
```

Figure 6: Instruction MMU Request Mismatch

5.1 Struggles

An initial struggle was selecting cores for the system to oversee. Originally, we had wanted to implement cores with 5-stage pipelines and more in-depth branch prediction features from different vendors, however finding two of these open source cores proved to be impossible. At first, we switched to using a Chipyard[7] generated core in addition to another core from the Core-V project. However, these cores did not have matching bus protocols, which would result in extremely complex methods to try and unpack the two differently. This eventually led to the pivot towards the S-core and X-core.

The largest, struggle throughout development was architectural verification. While the Core-V Verification Strategy was useful, both the S-core and X-core have migrated to a UVM based variant of the verification environment. Lacking a simulator for this, we were forced to use the depreciated test environment that supports Verilator. However, this also mandated that we use older core versions to match these test benches. Despite both the core and environments being on the correlated versions, there were still massive issues with pin naming discrepancies, S-core security features being incorrectly driven (or ignored), the environment requiring specific Verilator versions and more.

Additionally, debugging system level issues when testing the full combined architecture proved challenging, as tracing any number of values triggered proved incredibly time consuming. Overall, architectural verification was the largest hurdle during development, and for future implementations of the system should be prioritized knowing this.

5.2 Areas for Improvement

While our system does provide oversight for the two monitored IPs, it introduces several cycles of latency, and reduces the throughput of the monitored based on the slowest core. This works for our proof of concept, but in real life situations introduces a tradeoff between speed and security. Additionally, as implemented our system only works using matching bus protocols. For differing bus protocols, it's required for an implementer to make some sort of translator IP that unifies the formats of requests.

5.3 Notes on Recreation

Be aware that modifications were made to the Core-V Verification Strategy, and both cores are using version 0.9.0. Commands to apply these modifications and switch to these versions can be found in the makefile within the main directory, and are applied automatically when running setup.sh.

5.4 Labor Division

Component	Contributors
Comparison Unit Design	Kristian
External MMU Design	Jayden
Reset Buffer Design	Jayden
FIFO Design	Kristian
Top Module Design	Kristian
Architectural Verif.	Jayden & Kristian
Functional Verif.	Jayden
Security Verif.	Jayden
System Level Testing	Kristian

Table 2: Labor Division of the project

6 References

[1] "OpenHW Group — OpenHW Foundation," 2026. https://openhwfoundation.org/

[2] "OpenHW Group CV32E40X User Manual — CORE-V CV32E40X User Manual documentation," Openhwgroup.org, 2020. https://docs.openhwgroup.org/projects/cv32e40x-user-manual/en/latest/index.html

[3] "OpenHW Group CV32E40S User Manual — CORE-V CV32E40S User Manual documentation," Openhwgroup.org, 2020. https://docs.openhwgroup.org/projects/cv32e40s-user-manual/en/latest/index.html

[4] Anthropic, "Claude," Claude, 2026. https://claude.ai/share/9b2d627d-187e-457c-8969-77e622ef09ac

[5] "OpenHW Group CORE-V Verification Strategy — CORE-V Verification Strategy documentation," Openhwgroup.org, 2020. https://docs.openhwgroup.org/projects/core-v-verif/en/latest/index.html#

[6] A. Waksman and S. Sethumadhavan, "Tamper Evident Microprocessors." Available: https://www.cs.columbia.edu/~simha/cal/pubs/pdfs/TEMP_Oakland10.pdf

[7] "Welcome to Chipyard's documentation (version 'v1.11.0')! — Chipyard v1.11.0 documentation," chipyard.readthedocs.io. https://chipyard.readthedocs.io/en/stable/